Pipelined MIPS Processor in VHDL

Design, Instruction Encoding, and Python Assembler

IMPLEMENTATION

5-Stage Pipeline · VHDL

TOOLCHAIN

Python Assembler · ModelSim

ISA

MIPS Subset · 19 Instructions

Team Members

Ibrahim Wael

Abdelrahman Shehata

Abdullah Ahmed

Youssef Walid

Under Supervision of **DR. Hatem Zakaria**

Project Goals

① **Build a 5-Stage Pipeline**

Implement a full pipelined 32-bit MIPS processor in VHDL with IF, ID, EX, MEM, and WB stages

② **Support a Practical ISA Subset**

Implement 19 MIPS instructions across R-type, I-type, and J-type formats

③ **Load Programs via Instruction Memory**

Use imem.vhd as a 64-word ROM, initialized with machine code in hex format

④ **Python Assembler Bridge**

Automate assembly → machine code translation with asm2hex.py, producing VHDL-ready output

High-Level Architecture



← Forwarding Unit (EX/MEM → EX, MEM/WB → EX)

Key Modules

Top entity: mips_pipeline (TopLevel.vhd)

PC Register	Program counter, stall support	**Instruction Memory**	64-word ROM (imem.vhd)
Register File	32 regs, 2 read / 1 write ports	**ALU + ALU Control**	Arithmetic & logic operations
Data Memory	Load / store (dmem.vhd)	**Forwarding Unit**	EX/MEM & MEM/WB → EX forwarding
Hazard Detection	Load-use stall + control flush	**Main Control**	Decodes opcode → control signals

Pipeline Stages — What Happens Each Cycle

IF

Instruction Fetch

- Fetch 32-bit instruction at current PC
- Increment PC ← PC + 4
- Store PC+4 into IF/ID pipeline register
- Stall PC on load-use hazard

ID

Instruction Decode

- Decode opcode and funct fields
- Read two source registers (rs, rt)
- Sign- or zero-extend 16-bit immediate
- Compute branch/jump target addresses

EX

Execute

- ALU performs operation (add, sub, and, or…)
- Forwarding mux selects ALU operands
- Branch condition evaluated (zero flag)
- Jump/JAL/JR resolved here

MEM

Memory Access

- lw: read 32-bit word from data memory
- sw: write register to data memory
- Non-memory instructions pass through
- Address = ALU result from EX stage

WB

Write Back

- Select result: ALU output or memory read
- Write result to destination register
- JAL writes PC+4 to $ra ($31)
- LUI writes shifted immediate

Key VHDL Modules

Module	File	Role
Program Counter	pc_reg.vhd	32-bit PC with stall support
Instruction Memory	imem.vhd	64-word ROM, loaded with hex machine code
Pipeline Registers	ifid / idiex / ieximem / imemiwb	Latch stage outputs; handle flush & stall
Register File	reg_file.vhd	32 registers, 2 read ports, 1 write port
ALU	alu.vhd	AND, OR, ADD, SUB, XOR, NOR, SLT operations
ALU Control	alu_control.vhd	Translates ALUOp + funct ⟶ ALU operation
Main Control	main_control.vhd	Opcode/funct ⟶ full control signal bus
Data Memory	dmem.vhd	Synchronous read/write data memory
Forwarding Unit	forwarding_unit.vhd	EX/MEM and MEM/WB forwarding to EX stage
Hazard Detection	Hazard_detection.vhd	Load-use stall + control hazard flush
Top Level	TopLevel.vhd	Structural integration of all components

Supported Instruction Set — 19 Instructions

R-TYPE	8
Opcode = 0	*Distinguished by funct field*

add	sub	and
or	xor	nor
slt	jr	

I-TYPE	9
Various opcodes	*16-bit immediate operand*

addi	slti	andi
ori	lui	lw
sw	beq	bne

J-TYPE	2
Opcode 2 / 3	*26-bit word address*

j	jal

⚠ **NOT full MIPS — no multiply, divide, or shift instructions in this design**

Instruction Formats — 32-bit Word Layout



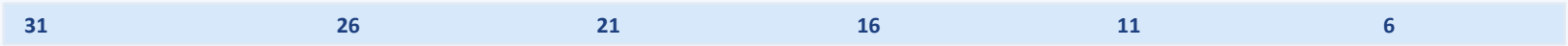
Format: op(6) | rs(5) | rt(5) | rd(5) | shamt(5) | funct(6) → Destination = rd, shamt=0 for non-shifts



Format: op(6) | rs(5) | rt(5) | imm(16) → Destination = rt, immediate sign-extended to 32 bits



Format: op(6) | target_address(26) → Full PC = {PC+4[31:28], addr[25:0], 00}



← Bit positions (MSB to LSB) →

Worked Encoding Example: addi $t0, $0, 5 → 0x20080005

Step 1 Identify Instruction Type

```
addi  →  I-type
Format: op(6) |
rs(5) | rt(5) |
imm(16)
```

Step 2 Opcode Field

```
addi opcode  =  8
→  binary: 001000
```

Step 3 Register Fields

```
rs = $0 (source) = 0
→  00000
rt = $t0 (dest)   =
8  →  01000
```

Step 4 Immediate Field

```
imm = 5  →  0000
0000 0000 0101
```

Step 5 — Concatenate All Fields:

op	rs	rt	immediate
001000	00000	01000	0000000000000101

Binary: **0010 0000 0000 1000 0000 0000 0000 0101** → Hex: **0x20080005**

This is the core logic implemented in asm2hex.py — field extraction and bit-packing for every instruction.

Opcode Reference Table

Type	Instruction(s)	Opcode (dec)	Opcode (hex)	Notes
R	add, sub, and, or, xor, nor, slt	0	0x00	funct 0x20–0x27, 0x2A
R	jr (jump register)	0	0x00	funct = 0x08
I	addi	8	0x08	Sign-extended immediate
I	slti	10	0x0A	Set less than immediate
I	andi, ori	12, 13	0x0C, 0x0D	Zero-extended immediate
I	lui	15	0x0F	Upper 16 bits → rd[31:16]
I	lw, sw	35, 43	0x23, 0x2B	Base + sign-extended offset
I	beq, bne	4, 5	0x04, 0x05	Branch offset in words
J	j (jump)	2	0x02	PC={PC+4[31:28],addr,00}
J	jal (jump-and-link)	3	0x03	Writes PC+4 → $ra ($31)

Control Unit & Hazard Handling

Main Control Signals

- RegDst — select rd vs rt
- ALUSrc — register vs immediate
- MemToReg — memory or ALU result
- RegWrite — enable register write
- MemRead / MemWrite
- Branch / BNE — conditional jump
- Jump / JAL / JR — unconditional
- LUI — upper immediate load
- Imm ZeroExt — zero vs sign extend

Hazard Detection Unit

- Load-Use Hazard: detect lw → dependent instruction
- Action: stall PC, freeze IF/ID
- Insert bubble (NOP) into ID/EX
- One-cycle stall per load-use pair
- Control Hazard: branch/jump taken
- Flush IF/ID and ID/EX registers
- Discards 2 instructions in pipeline

Forwarding Unit

- EX/MEM → EX: forward result one cycle ahead
- MEM/WB → EX: forward result two cycles ahead
- Priority: EX/MEM forwarding over MEM/WB
- Covers: R-type, addi, and most ALU ops
- Does NOT forward after lw (→ stall instead)
- Mux selects between forwarded and register data

ALU Operations

AND — ALUOp: 000

`$t0 & $t1`

Bitwise AND — andi uses zero-extended imm

OR — ALUOp: 001

`$t0 | $t1`

Bitwise OR — ori uses zero-extended imm

ADD — ALUOp: 010

`$t0 + $t1`

32-bit addition — addi, lw, sw

XOR — ALUOp: 100

`$t0 ^ $t1`

Bitwise exclusive-OR

NOR — ALUOp: 101

`~($t0|$t1)`

Bitwise NOR — NOT(rs OR rt)

SUB — ALUOp: 110

`$t0 - $t1`

Subtraction — drives beq/bne zero flag

SLT — ALUOp: 111

`$t0 < $t1 ? 1 : 0`

Set-less-than (signed) — slti, slt

LUI (Load Upper Immediate): bypasses ALU — immediate shifted left 16 bits, lower 16 bits zeroed. Branches use SUB result: zero flag → beq taken; non-zero → bne taken.

Python Assembler — asm2hex.py

Input	MIPS .asm file with labels, pseudo-ops, comments (#)
Pass 1	Scan labels → build symbol table (label → word address)
Pass 2	Encode each instruction using opcode/funct tables
Output	VHDL mem_array constants ready for imem.vhd
Pseudo-ops	nop (add $0,$0,$0), move (add rd,rs,$0), li (addi + lui)
Usage	python asm2hex.py program.asm --vhdl

asm2hex.py — core logic

```
# Pass 1: collect labels
for line in src:
  if ':' in line:
    labels[name] = pc
  else: pc += 1

# Pass 2: encode
for inst in parsed:
  op = OPCODES[inst[0]]
  rs = REGS[inst[1]]
  rt = REGS[inst[2]]
  imm = int(inst[3])
  word = (op<<26)|(rs<<21)
       |(rt<<16)|imm
  output.append(word)
```

Example Program: Factorial (n=5 → 120)

program.asm

```
# Factorial n=5 → $t1
        addi  $t0, $0, 5    # n=5
        addi  $t1, $0, 1    # result=1
loop:   slt   $t2, $0, $t0 # 0 < n?
        beq   $t2, $0, done
        add   $t1, $t1, $t0
        addi  $t0, $t0, -1 # n--
        j     loop
done:   sw    $t1, 0($s2)  # save
        j     done         # halt
```

Generated Machine Code (imem.vhd)

addi	$t0,$0,5	0x20080005
addi	$t1,$0,1	0x20090001
slt	$t2,$0,$t0	0x0008502A
beq	$t2,$0,done	0x114B0004
add	$t1,$t1,$t0	0x01284820
addi	$t0,$t0,-1	0x2108FFFF
j	loop	0x08000002

Expected Pipeline Result

After 10 cycles:

- $t1 = 120 (5! = 5×4×3×2×1)
- dmem[0] = 120 (sw writes result)
- PC loops at 'done' (halt loop)
- Forwarding handles loop RAW hazards

Simulation Workflow — Assembly to Waveform

1 Write Assembly

`program.asm`
MIPS instructions + labels + comments

2 Run Assembler

`python asm2hex.py program.asm --vhdl`

3 Paste into imem.vhd

Copy `mem_array` constants into `imem.vhd` ROM

4 Simulate

ModelSim: compile `mips_pipeline` + `tb.vhd` testbench

5 Verify Results

Inspect registers, memory, PC in waveform viewer

Verification Checklist

- ✓ **ALU Result**
  Verify $rd/$rt gets correct arithmetic result after WB stage
- ✓ **Load/Store**
  Check dmem reads and writes correct 32-bit value at address
- ✓ **Stall**
  Confirm 1-cycle bubble inserted after lw ⟶ dependent instruction
- ✓ **Branch Targets**
  Confirm PC jumps to correct label on beq/bne condition
- ✓ **Forwarding**
  Ensure no stale register value passed to ALU when forwarding active
- ✓ **JAL**
  Verify $ra ($31) = PC+4 and PC jumps to target

Results & Limitations

✅ Results Achieved

- Fully functional 5-stage pipelined MIPS processor
- EX/MEM and MEM/WB forwarding eliminates most stalls
- Load-use hazard detection with 1-cycle stall insertion
- Control hazard flush on taken branches and jumps
- 19-instruction subset covering core MIPS operations
- Automated assembler (asm2hex.py) with two-pass label resolution
- VHDL output directly loadable into imem.vhd ROM
- ModelSim simulation validates all major execution paths

⚠️ Known Limitations

No cache

Direct memory access — no L1/L2 cache hierarchy

No coprocessor

No CP0, exceptions, or interrupt handling

No shifts

sll, srl, sra not implemented (shamt unused)

No multiply/divide

mul, div, mfhi, mflo absent

Flush, not predict

All taken branches cause 2-cycle penalty

Fixed IMEM size

64-word instruction ROM; no dynamic loading

Conclusion & Future Work

Summary

A complete, functional teaching-grade MIPS pipeline implemented in VHDL, accompanied by a custom two-pass Python assembler. Demonstrates all key pipelining concepts: forwarding, hazard detection, branch/jump handling, and control signal generation across all five classic RISC pipeline stages.

Future Work

⊕

Extend ISA

Add shift instructions (sll/srl/sra), multiply, divide

⊙

Branch Prediction

Static or dynamic 2-bit predictor to reduce flush penalty

⊛

CI Integration

Auto-assemble → simulate → check in continuous integration

⊞

C Compiler Front-end

Target the assembler from a minimal subset C compiler

Questions & Discussion — Thank you!